



Figure 4: Components of your own GitHub Action

GitHub Marketplace that you can use readily within your own workflows and build on top of.

Lastly, GitHub Actions is completely free for public repositories, and has free limits for private repositories as well.

Why build your own action?

While there are thousands of actions available on the GitHub Marketplace that you could use, there could be scenarios where you might have to create your own action extension for use within your workflows.

First, if you want to perform certain tasks or have some custom logic for which an action extension doesn't already exist on the Marketplace, it is often useful to write your own action. This helps make your workflow modular, and also enables reuse and sharing within your teams, projects or organisations.

Second, if you want to integrate GitHub and any other services/tools that you might use, you can do this easily by creating a GitHub Action that interacts with the APIs of those services/tools. This makes it easier for you to interact with other tools from within your workflows through GitHub Actions.

One of the main benefits of GitHub Actions being native to GitHub is

that it is accessible to the millions of developers using the latter. If an action that you create on your own can be used by other developers as well, you can publish it yourself for free on the GitHub Marketplace and make it available to everyone on GitHub.

How to build your own action

An action has essentially two components (Figure 4). One is a YAML file called `action.yml` and the other is the source code of your action.

Let us look at what exactly these two components do.

The `action.yml` file stores the metadata about the action you are building. It stores details like the name of the action, what are the inputs it needs and the outputs it delivers. It also

defines whether the source code of the action is in JavaScript that can be run using Node12, or is a Docker image. It also contains information about where to find the JavaScript files or the Docker image to run the action.

The source code part of your action contains the actual logic of it. This can be JavaScript files that can run using Node12, a Dockerfile that you are going to build when an action runs, or it can simply be a Docker image that you refer to. It will have access to the event payload that triggered the workflow and also the context of the workflow run through environment variables (ENV). You can easily call GitHub's APIs or other APIs from within your source code as a part of your logic, to accomplish the action's task.

GitHub Actions lets you create your own actions using the millions of open source libraries accessible on GitHub, and allows you to write them in JavaScript or create a container action. It makes software workflow automation easy, which helps you be more productive and eases your development life cycle.

You can automate various use cases by using GitHub Actions, including API management, code quality, support, chat, code review, publishing, deployment, localisation, continuous integration, learning, project management, monitoring, security, dependency management, and others. **END** 🐧

References

- [1] Understanding GitHub Actions: <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>
- [2] Creating custom actions: <https://docs.github.com/en/actions/creating-actions/about-custom-actions>
- [3] Actions on the GitHub Marketplace: <https://github.com/marketplace?type=actions>
- [4] Interactive learning on GitHub with Learning Lab: <https://lab.github.com/>

By: M.V. Karan

The author is a senior developer relations manager at GitHub. This article is based on a tech talk he gave at OSI 2021 (virtual).

Transcribed and curated by Laveesh Kocher.